

Architecture & Design Document:

1. Stack Choices & Justifications

- **Next.js (App Router):** Provides a unified, type-safe full-stack framework. Server Components reduce bundle size for data-dense tables, while Server Actions / Route Handlers streamline backend logic and backend-enforced authorization without maintaining a separate API repository.
- **Better-Auth (Email + Password):** Self-hosted, lightweight authentication configured with public sign-ups disabled. Integrates directly into database tables, handles session cookies securely on the server, and allows admin-only account creation out-of-the-box.
- **Prisma & PostgreSQL:** Relational PostgreSQL provides ACID guarantees required for booking concurrency. Prisma offers end-to-end type safety, reliable migrations, and clean transaction controls essential for race condition prevention.
- **Tailwind CSS & shadcn/ui:** Accessible, clean component primitives that allow rapid UI development without heavy overhead, ensuring an intuitive, no-training-needed experience for non-technical front-desk staff.

2. Core Design Decisions

- **Strict Backend Access Control (RBAC):**
 - **Admin:** User and role management only (no workshop editing or registration).
 - **Manager:** Workshop CRUD, register/cancel attendees, view history.
 - **Staff:** Register/cancel attendees, view catalogue & history (no workshop creation/editing).
 - *Enforcement:* Permission validation is executed in backend server functions/handlers via session verification before any query is dispatched, refusing unauthorized attempts regardless of UI state.
- **Auditability & Soft-State Registrations:**
 - Registrations are **never hard-deleted**. When cancelled, a status flag (**ACTIVE** → **CANCELLED**) is set alongside **cancelledAt** and **cancelledByUserId**.
 - Every registration persists **registeredByUserId** and **registeredAt** to satisfy the audit trail requirement.
- **Filtering & Search:**
 - Optimized multi-criteria queries on the backend allowing staff to filter simultaneously by date ranges, workshop status (**DRAFT**, **PUBLISHED**, **COMPLETED**), and active seat availability.

3. How Over-Registration is Prevented (Concurrency & Race Conditions)

On busy Saturday mornings when multiple staff members register attendees for the final seat simultaneously, naive read-then-write checks cause race conditions (overbooking).

To guarantee capacity limits hold under high concurrency:

1. **Database Transactions** : The registration attempt is wrapped inside an atomic transaction.
2. **Pessimistic Locking / Atomic Guard**:
 - *Row-Level Lock*: Uses PostgreSQL `SELECT ... FOR UPDATE` on the target workshop record to serialize concurrent booking attempts for the same workshop.
 - *Atomic Capacity Verification*: Within the lock, active registrations (`status = 'ACTIVE'`) are counted and compared against `workshop.capacity`.
 - *Conditional Write*: If `activeCount >= capacity`, the transaction immediately rolls back and throws a typed `WorkshopFullError`.
 - *Isolation*: PostgreSQL guarantees that only one transaction can acquire the seat; subsequent concurrent requests see the updated count immediately and fail cleanly with a user-friendly error message.

4. Trade-offs

- **Monolithic Next.js vs. Decoupled Services**: A single Next.js app simplifies deployment and shared TypeScript types across API and UI within a 3-hour window, at the cost of coupling API scaling directly to the web frontend.
- **Computed Seat Availability vs. Cached Counter Column**:
 - Counting `ACTIVE` rows inside the locked transaction guarantees 100% accuracy and eliminates counter desynchronization bugs.
 - *Trade-off*: Slightly higher query cost compared to a denormalized `remainingSeats` counter column, but completely eliminates drift for workshops with standard capacities (under 100 seats).
- **Server-Rendered Table Filters vs. Full Client-Side Grid**:
 - Used URL-driven search parameters and server querying rather than downloading all historical data into client memory. This keeps load times low even as registration history scales.

5. Assumptions Made

- **Attendee Identity**: Attendees do not have logins or accounts; duplicate emails across *different* workshops are permitted, but duplicate *active* registrations with the same email in the *same* workshop are rejected to avoid double bookings.
- **Location Scope**: While the business has three locations, location is stored as an attribute on the workshop record (or filter) rather than requiring multi-tenant branch isolation.
- **Seed Data**: An initial Admin account is provisioned via seed scripts to bootstrap the system, after which that Admin provisions Manager and Staff accounts.